



TC-YOLOv5: rapid detection of floating debris on raspberry Pi 4B

Shun Li¹ · Shubo Liu¹ · Zhaohui Cai¹ · Yuan Liu¹ · Geng Chen¹ · Guoqing Tu²

Received: 3 July 2022 / Accepted: 11 October 2022 / Published online: 13 February 2023
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2023

Abstract

Floating debris is a prominent indicator in measuring water quality. However, traditional object detection algorithms cannot meet the requirement of high accuracy due to the complexity of the environment. It is difficult for some deep learning-based object detection algorithms to achieve fast detection due to the limited performance of embedded devices. To address the above issues, this paper proposes TC-YOLOv5, which improves the detection accuracy by integrating the convolutional block attention module and vision transformer. To ensure the efficiency and low resource consumption of the algorithm, we selectively remove some convolutional layers and reduce some redundant calculations. We evaluated the performance of TC-YOLOv5 on a dataset with multiple species of floating debris, which can process an image in an average of 1.18 s on a Raspberry Pi 4B and achieve the mean average precision (mAP@0.5) of 84.2%. The detection accuracy, speed, and floating-point operations (FLOPs) of TC-YOLOv5 are better than some algorithms of the YOLOv5 series, such as YOLOv5n, YOLOv5s, and YOLOv5m. The above data show that TC-YOLOv5 realizes high-precision, low resource consumption, and rapid detection.

Keywords Deep learning · Floating debris detection · YOLOv5 · Raspberry Pi · Attention module

1 Introduction

With the improvement of human living standards, the public awareness of environmental protection is increasing [22]. To better monitor the water quality, object detection technology has been introduced to the field of floating debris detection, which has significant commercial value in civil fields, such as unmanned surface vehicles (USVs) [2] and unmanned aerial vehicles (UAVs) cruises. Although object detection techniques have achieved good results in various tasks, there is still a need for improvement in the detection of floating debris, especially when applied to performance-constrained embedded devices such as Raspberry Pi, where a balance of speed and accuracy needs to be considered.

The basic principle of the traditional floating debris detection method is to first select the area in the image where floating debris may appear, then use artificially designed

extractors for feature extraction, such as SIFT [20] and HOG [3], and finally use SVM [12] classifiers to classify the features collected in the previous step. The artificially designed extractor is unsuitable for identifying floating debris in complex waters because it contains fewer parameters, has lower robustness, and is susceptible to factors, such as the variety of detected objects and the complexity of the environment.

With the rapid development of deep learning, computer vision technology has made remarkable achievements, especially the basic branch of object detection, and numerous classical algorithms have emerged, which have been widely used in various fields, such as autonomous driving [5], intelligent medical [31] and intelligent security [15]. Compared with traditional object detection algorithms, the large number of parameters of deep neural networks can extract features with better robustness and semantics, and the classifier performance is superior. There are two main types of deep learning-based object detection algorithms: two-stage object detection algorithms and single-stage object detection algorithms. The basic principle of two-stage object detection is that the algorithm first generates a series of candidate boxes of some objects, and then classifies the object by convolutional neural networks, such as R-CNN [6], Fast R-CNN [7] and Faster R-CNN [27]. Single-stage object detection

✉ Shubo Liu
liu.shubo@whu.edu.cn

¹ School of Computer Science, Wuhan University, Wuhan 430072, China

² School of National Cybersecurity, Wuhan University, Wuhan 430072, China

algorithms don't need to generate object candidate boxes, but directly transform the object box localization problem into a regression problem, such as SSD [19] and YOLO series [1, 8, 24–26]. YOLO is very suitable for practical applications because of its fast detection speed.

Some researchers have used deep learning-based object detection algorithms to detect floating debris. Latif et al. [14] realized the detection and counting of floating debris based on the YOLOv4 algorithm, which improves the efficiency of counting, but the speed of detection is slow due to the high complexity of the YOLOv4 algorithm; Paller and Éllő [21] proposed two iterations of early warning system for floating plastic waste. The first iteration uses motion detection to identify relevant images, which is easy to implement but is only detected when the floating garbage forms a larger block of debris. The second iteration uses the Faster R-CNN algorithm for detection, which has better detection accuracy than the first iteration, but it requires the support of GPU.

Embedded devices are eminently suitable for deploying practical application algorithms because of their low cost, low power consumption and small size. Some researchers have applied the YOLO series algorithms on embedded devices to detect floating debris. Li et al. [16] introduced the two-scale detection YOLOv3 algorithm into the water surface garbage capture robot with graphics cards to achieve real-time detection and capture of the water surface garbage. Zailan et al. [32] used the YOLOv3 algorithm to detect floating garbage and pass the detected location to the water surface cleaner robot for floating garbage collection. While both of these efforts can achieve good results, they require the support of graphics cards or edge devices, which brings high costs and complex deployment steps. To alleviate the above problems and relate to practical application scenarios, we train the detection model of floating debris on the computer and deploy the trained deep learning model on the Raspberry Pi 4B to perform the detection task.

Based on the application scenarios of embedded devices and the characteristics of floating debris, we propose an improved YOLOv5 algorithm. As a state-of-the-art detector, YOLOv5 has the advantages of high accuracy and high speed. YOLOv5n is the fastest algorithm in the YOLOv5 series, which means it can be easily ported to mobile devices. These advantages are beneficial for ensuring the detection precision and speed of the floating debris detection. Then, we further improve YOLOv5n and propose TC-YOLOv5, which has high speed and is reliable for detecting floating objects on the water surface. In the design of the TC-YOLOv5 network, on the one hand, we add a convolutional block attention module (CBAM) [30] to the back of the bottleneck block to enhance the learning capability of the network. CBAM is a lightweight attention module that doesn't change the size and number of channels of the input feature map, so it is easy to add them to the YOLOv5n algorithm;

on the other hand, we integrate vision transformer [4] into the last Cross-Stage-Partial-connections (CSP) [28] module of the backbone network. Vision transformer can help the network capture abundant global and contextual information to extract more powerful features. At the same time, some convolutional layers are selectively removed, and some redundant computations are reduced to ensure the inference speed of the model. Our key contributions are as follows:

1. We remove some convolutional layers and reduce the amount of redundant computation in some layers to improve the detection speed.
2. We add CBAM and the vision transformer to the CSP module of YOLOv5n, which helps the network to focus on the key information while getting the global information.
3. We deploy the TC-YOLOv5 model on Raspberry Pi 4B, which can detect floating debris in an image in an average of 1.18 s.

2 Related work

2.1 YOLOv5 object detection algorithm

YOLO is a single-stage classical detector. Compared with the two-stage structure of Faster R-CNN, YOLO creatively uses the first-order structure to complete the object detection task, transforming the object box localization problem into a regression problem processing, directly predicting the class and location of the object without using the pre-selected box similar to anchor, with fast prediction speed, and is widely used in industry.

YOLOv5 is the latest achievement of the YOLO series, which not only inherits the advantages of the previous version, but also absorbs some advantages of other networks, so it is better than all previous versions in terms of accuracy and speed. YOLOv5 adjusts the depth and width of the network through the `depth_multiple` and `width_multiple` parameters, `depth_multiple` adjusts the depth of the network by controlling the number of bottlenecks, and `width_multiple` adjusts the depth of the network by controlling the number of convolutional kernels. In 2021, Ultralytics released version 6.0 of YOLOv5, proposing a new network, YOLOv5n, which is faster than YOLOv5s, YOLOv5m, YOLOv5l and YOLOv5x, so it is very suitable for rapid object detection. In this paper, the detection model is based on YOLOv5n, which can be divided into Backbone, Neck and Head, and its network structure is shown in Fig. 1a.

YOLOv5 model mainly consists of CBS, CSP, SPPF, Upsample and Concat. CBS is a convolutional block that contains Conv, Bn and SiLU, as shown in Fig. 1b. The CSP structure of YOLOv5 divides the original input into

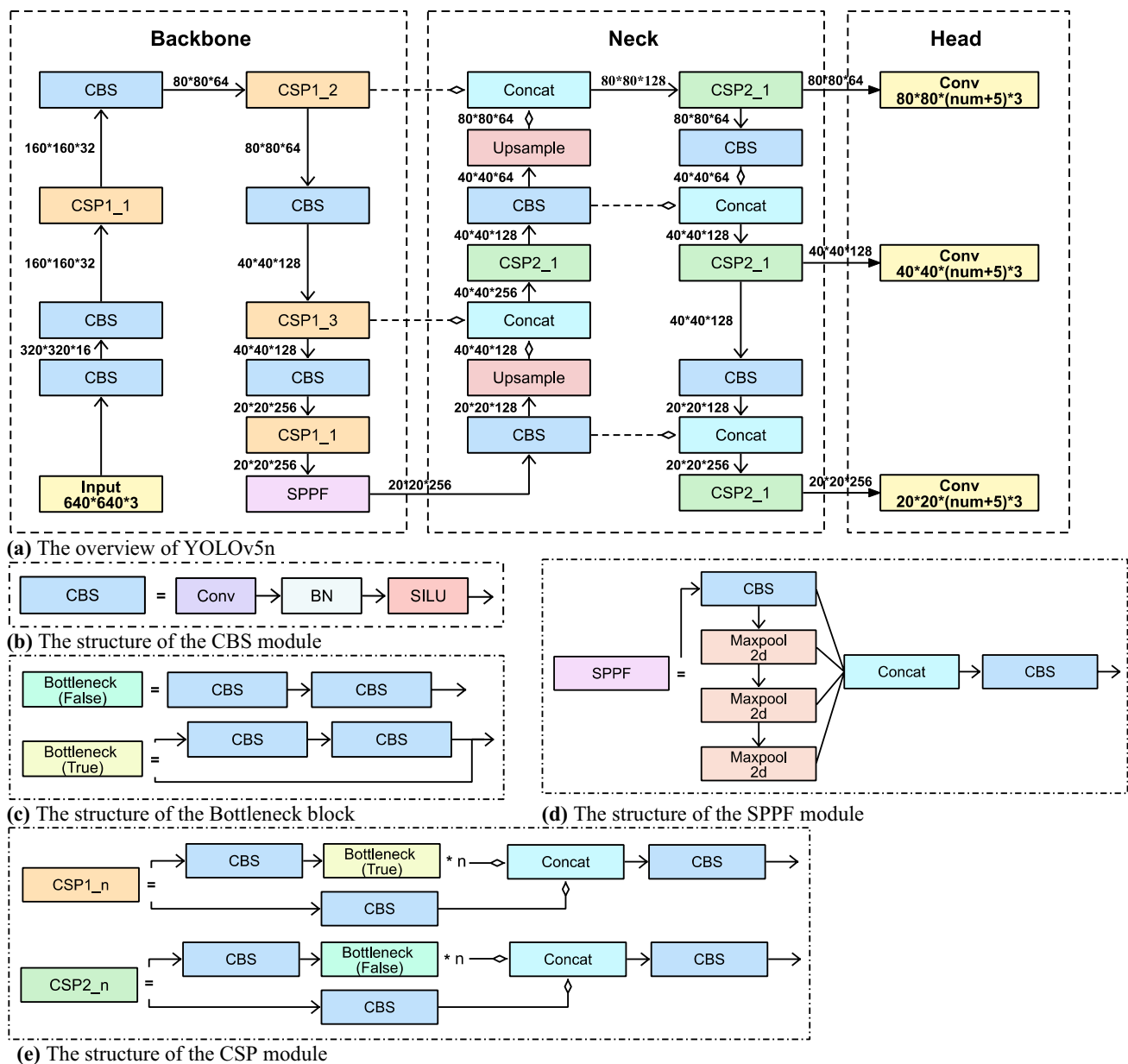


Fig. 1 The schematic illustration for YOLOv5n

two branches. It performs the convolution operation separately to halve the number of channels, one of the branches continues the bottlenecks operation and the other branch remains unchanged, and then stacks the two branches along the channel dimension, and finally outputs the result after 1D convolution. This structure enables the model to learn more features. YOLOv5 has two different CSP structures, as shown in Fig. 1e. CSP1_n module uses a bottleneck block with a residual structure, mainly used in the Backbone. The Backbone is a deeper network, so adding the residual structure can increase the gradient value of the backpropagation between layers, avoiding the gradient disappearance due

to network deepening, so that finer-grained features can be extracted without worrying about the network degradation. CSP2_n module uses a bottleneck block without residual structure, mainly used in the Neck. The bottleneck block is mainly composed of two convolutional blocks, and it is determined whether it has a residual structure according to the passed parameters, as shown in Fig. 1c. YOLOv5 uses Spatial Pyramid Pooling—Fast (SPPF) instead of SPP [10]. The SPPF module consists of four parallel branches, as shown in Fig. 1d. It uses the maximum pooling method, which effectively avoids problems, such as image distortion caused by cropping and scaling operations on images, and

improves the connection method of pooling, which effectively reduces the computation.

The input side of YOLOv5 uses Mosaic data augmentation, adaptive anchor box calculation and adaptive image scaling. The principle of Mosaic data augmentation is to stitch four images using random scaling, random cropping and random arrangement, which enriches the dataset and reduces the GPU usage during training. In the YOLOv5 algorithm, some anchor boxes are preset for different datasets. In the network training, the network outputs prediction boxes based on the initial anchor boxes, then compares them with the actual boxes, calculates the difference value between them, and then updates them in reverse to iterate the network parameters. The calculation of the initial anchor boxes is realized by the k-means clustering algorithm. The adaptive image scaling method improves the inference speed of the algorithm by adaptively adding the smallest black edge to the original image.

To make full use of the features extracted by the network at different levels, feature pyramid networks (FPN) [17] and pyramid attention network (PAN) [18] are used in the Neck layer of YOLOv5, as shown in Fig. 2. Since the deep feature map has stronger semantic features and weaker localization information, the shallow feature map has stronger location information and weaker semantic features. The role of FPN is to pass the semantic features from the deep layer to the shallow layer, thus enhancing the semantic representation at multiple scales. The role of PAN is to pass the localization information from the shallow layer to the deep layer, enhancing the localization capability at multiple scales.

2.2 Attention mechanism

Humans can effectively determine the important regions in an image. Motivated by this observation, attention mechanisms were introduced into computer vision with the aim of imitating this aspect of the human visual system [9]. The attention mechanism gives the neural network the ability to focus on a subset of its inputs, allowing the model to learn some of the key features in the image. SENet [13] first proposed channel attention. The core of SENet is a squeeze-and-excitation (SE) block for collecting global information

and improving representation ability. Since SENet reduces the model complexity by reducing the number of channels, this reduces the detection accuracy of the model. ECANet [29] proposed the efficient channel attention (ECA) block, which uses 1D convolution instead of dimensionality reduction, significantly improving the performance of the model with the introduction of few parameters. BAM [23] uses dilated convolution to expand the receptive field of the spatial attention sub-module and builds a bottleneck structure as recommended by ResNet [11] to save computational costs. CBAM stacks channel attention and spatial attention in series, it emphasizes useful channels as well as enhancing informative local regions. CBAM is a lightweight module that can be seamlessly integrated into any CNN architecture with negligible additional cost. Vision transformer is the first pure transformer architecture for image processing, which demonstrated that a pure attention-based network can achieve comparable results to modern convolutional neural networks. We integrate the CBAM and vision transformer in YOLOv5n, which improved the accuracy of the model with a small amount of additional computation and inference time.

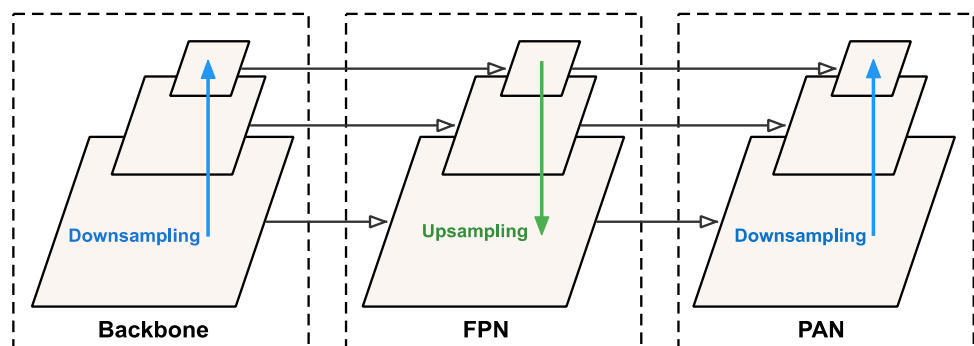
3 Methods

3.1 TC-YOLOv5

To simultaneously improve the detection accuracy and inference speed of the model, we make some improvements based on YOLOv5n, the main improvement is to replace CSP1_n and CSP2_n with C_CSP and TR_CSP. On the one hand, the C_CSP module only contains a bottleneck block, which reduces the computation of the network; on the other hand, the integration of the C_CSP module and TR_CSP module in the Backbone can improve the ability of the network to extract features; meanwhile, placing the C_CSP module in front of the Head layer can improve the accuracy of detecting floating objects on the water surface. The specific network structure of TC-YOLOv5 is shown in Fig. 3.

Woo et al. [30] proposed CBAM and integrated it with a ResBlock of ResNet. Inspired by this, we try to add the

Fig. 2 The structure of FPN and PAN in YOLOv5



CBAM to the back of the bottleneck block of the CSP module, which is C_CSP. The C_CSP module is a CSP module integrated with CBAM that enables the network to focus on regions of the image that contain more important features, the structure of which is shown in Fig. 4. The C_CSP module divides the original input into two branches, both branches perform convolution operations to halve the number of channels, one branch remains unchanged, and the other branch passes through the bottleneck block without residual structure and CBAM attention in turn, then the two branches are combined along the channel dimension, and finally the 1D convolution operation is performed to generate the output.

Zhu et al. [33] integrated the transformer into YOLOv5x by replacing some convolutional blocks and bottleneck

blocks with three transformer blocks, which can accurately localize objects in high-density scenes. We replace the bottleneck block in the CSP module with a transformer block, which is TR_CSP that helps the network extract more important features, and its structure is shown in Fig. 5. The TR_CSP module differs from the C_CSP module in that we use the replacement method instead of adding the transformer block to the back of the bottleneck block. This is because after experiments, adding the transformer block to the bottleneck doesn't significantly improve the accuracy of the model, but decreases the detection speed of the model. The replacement method can minimize the speed drop of the model. Similarly, the principle of using only one transformer block is to ensure the rapid detection of the model. Vision transformer uses a self-attention mechanism to capture

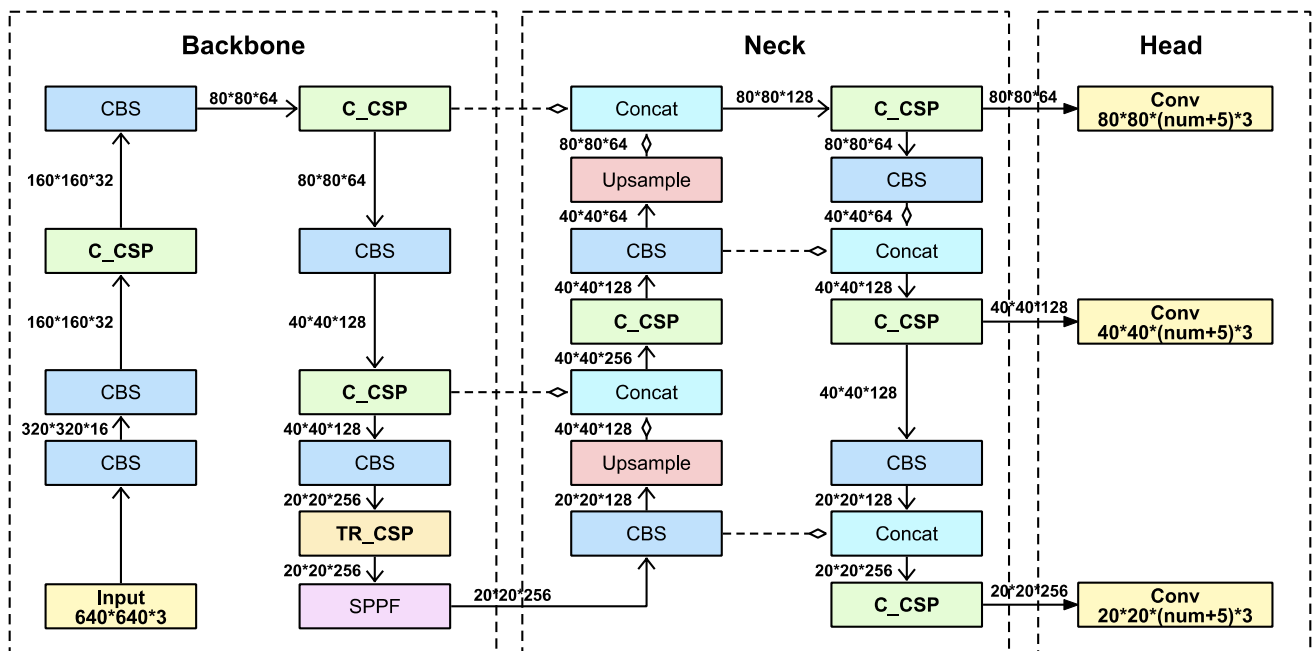


Fig. 3 The network structure of TC-YOLOv5

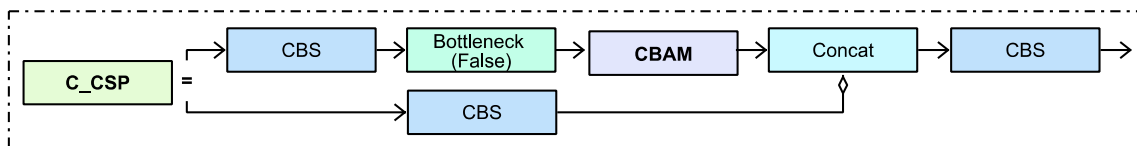


Fig. 4 The structure of the C_CSP module

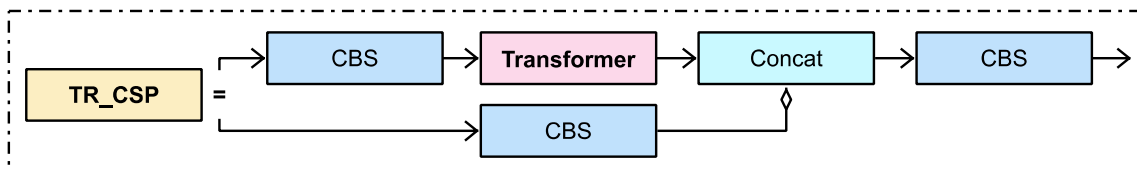


Fig. 5 The structure of the TR_CSP module

global contextual information, enabling the network to extract more powerful features. As the network deepens, the feature map at the end of the backbone network has become very small, and the use of the transformer block can achieve better results than the bottleneck block.

3.2 Convolutional block attention module

Convolutional block attention module (CBAM) contains two independent sub-modules: the channel attention module and the spatial attention module, as shown in Fig. 6. Given an intermediate feature map F , the attention weights are inferred along the channel dimension (M_c) and spatial dimension (M_s) in turn, and then multiplied with the original feature map to adaptively refine the features. Equation (1) is the mathematical representation of the overall attention process.

There are generally feature outputs of multiple channels in the convolution, and the features of some channels will

have a more significant impact on the final object detection result, so it is necessary to better focus on these channels, which is the channel attention, and it is generally achieved by pooling. As shown in Fig. 7, in the channel attention module, maximum pooling and average pooling are used respectively, and the nonlinear feature changes are carried out through the fully connected network. The changed network is summed and activated to obtain the attention weight. Its mathematical representation is shown in Eq. (2). The spatial attention enables the model to know where features are meaningful. It enables the model to learn more important features in space by allowing it to strengthen or weaken its focus on features at certain locations in space, mainly achieved by connecting different channels after maximum pooling and average pooling. Its structure is shown in Fig. 8, and the mathematical representation is shown in Eq. (3).

$$\begin{aligned} F' &= M_c(F) \otimes F \\ F'' &= M_s(F') \otimes F', \end{aligned} \quad (1)$$

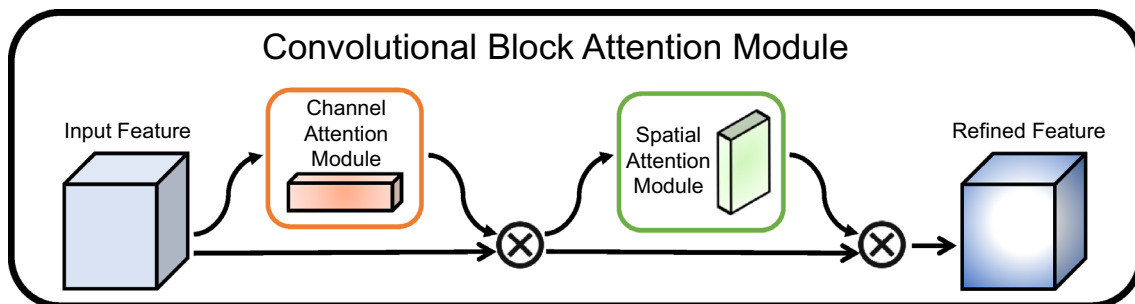


Fig. 6 The structure of CBAM

Fig. 7 The diagram of Channel Attention Module

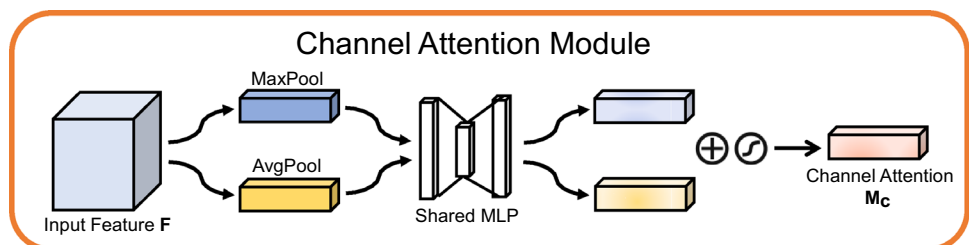
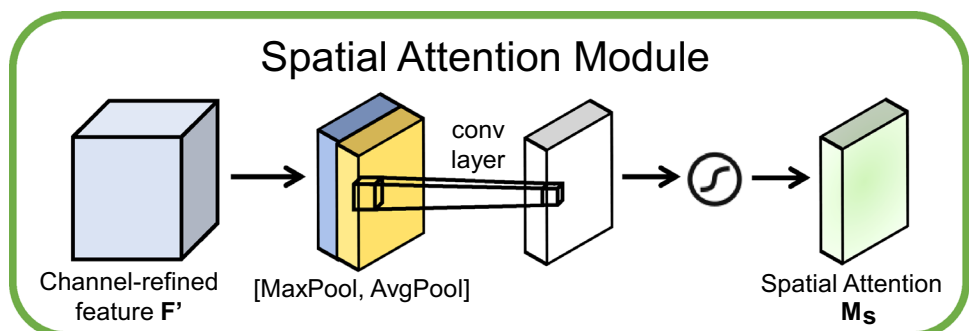


Fig. 8 The diagram of Spatial Attention Module



$$M_c(F) = \sigma(\text{MLP}(\text{AvgPool}(F)) + \text{MLP}(\text{MaxPool}(F))) \\ = \sigma(W_1(W_0(F_{avg}^c) + W_1(W_0(F_{max}^c))), \quad (2)$$

$$M_s(F) = \sigma(f^{7 \times 7}([\text{AvgPool}(F); \text{MaxPool}(F)])) \\ = \sigma(f^{7 \times 7}([F_{avg}^s; F_{max}^s])), \quad (3)$$

where $\otimes$ represents element-wise multiplication, σ means the sigmoid function, W_0 and W_1 are the weights of MLP, and 7×7 denotes the size of the convolution kernel.

The CBAM is a lightweight general-purpose module that can be integrated into existing network architectures as a plug-and-play module with negligible overhead and can be trained end-to-end along with a basic CNN. The optimization model proposed in this paper integrates CBAM, which allows the network to pay more attention to areas containing important features and suppress irrelevant information, thus improving the accuracy of object detection.

3.3 Vision transformer

The transformer was first applied to natural language processing with good results. Vision transformer is an application of the transformer model architecture to the field of computer vision, enabling the network to achieve good results in tasks, such as image classification and object detection, without relying on convolution. Convolutional neural networks use convolutional kernels to extract abstract high-level features, but still only capture local feature information in most cases.

The advantage of the transformer lies in the way it uses attention to capture global contextual information to establish a distant dependency on the object, thus extracting more powerful feature information. The transformer encoder mainly consists of a feed-forward neural network (MLP), multi-headed attention and some Norm operations, as shown in Fig. 9.

4 Experiments

4.1 Image datasets

The dataset of floating debris used in this paper is from CSDN (<https://www.csdn.net>), with 2208 images divided into training sets and validation sets according to a 3.6:1 ratio, of which 1727 are training sets and 481 are verification sets. The dataset includes seven categories of floating

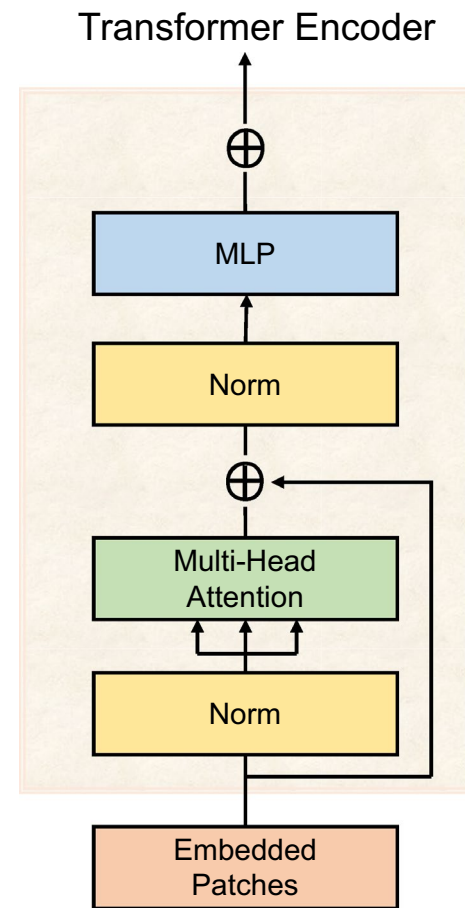


Fig. 9 The architecture of transformer encoder

Table 1 The number of labels for various categories of floating debris

Class	Number of labels
Bottle	1691
Branch	434
Milk-box	255
Plastic-bag	411
Plastic-garbage	202
Ball	49
Leaf	267
Total	3309

debris: bottle, branch, milk box, plastic bag, plastic garbage, ball, and leaf, with the number of tags in each category shown in Table 1. We enriched the dataset using mosaic data augmentation, which scaled and cropped the combined images and then randomly stitched them together.

4.2 Experimental platforms

This paper has two experimental platforms: a computer and a Raspberry Pi 4B.

The models were trained on a computer with Ubuntu 18.04, Intel Xeon Silver 4110 CPU, Quadro P6000 graphics card, compute unified device architecture (CUDA) version 11.0, and deep learning framework is Pytorch 1.7.0.

The trained models were deployed on a Raspberry Pi 4B with Ubuntu 20.10 system version, BCM2711 CPU model, and deep learning framework is Pytorch 1.8.0.

4.3 Evaluation metrics

We evaluate the performance of the model using metrics, such as $mAP_{@0.5}$, $mAP_{@0.5:0.95}$, model size, number of parameters, FLOPs, FPS, and the time it takes for the model to detect an image on the Raspberry Pi 4B. FLOPs means floating-point operations, and 1 GFLOPs is equal to 10^9 FLOPs.

AP means the average precision of the model in a certain category. mAP is mean average precision, which means the mean AP of the model over all categories. Their calculation formulas are shown in Eqs. (4) and (5).

$$AP = \int_0^1 P dR \quad (4)$$

$$mAP = \frac{\sum AP}{N(class)} \quad (5)$$

$mAP_{@0.5}$ represents the mAP at an intersection over union (IoU) threshold of 0.5, and $mAP_{@0.5:0.95}$ represents the average mAP over different IoU thresholds (from 0.5 to 0.95, in steps of 0.05).

4.4 Ablation experiment

To illustrate the effectiveness of TC-YOLOv5 and the impact of the individual modules added, we first perform model pruning and then sequentially integrate the CBAM and transformer block. Table 2 shows the results of the

ablation experiments in model size, floating-point operations, $mAP_{@0.5}$, $mAP_{@0.5:0.95}$ and FPS_{GPU} .

As shown in Table 2. Method M1 is the pruned YOLOv5n. The size of the model is reduced by 170 KB and the floating-point operations are reduced by 0.4 GFLOPs, with an increase in speed and a small loss in accuracy. Method M2 integrates CBAM on the basis of method M1. Since CBAM is a lightweight attention mechanism, the model does not increase the floating-point operations after adding CBAM, and the size of the model increased by 34 KB.

But the $mAP_{@0.5}$ and $mAP_{@0.5:0.95}$ increased by 1.2% and 0.6%. Method M3 integrates the vision transformer on the basis of method M1. Because we replace the bottleneck block of the CSP module with a transformer block, the size of the model barely changed, while the $mAP_{@0.5}$ and $mAP_{@0.5:0.95}$ increased by 1.8% and 0.9%. Finally, we use the above three tricks. Except for speed, TC-YOLOv5 outperforms YOLOv5n in accuracy, model size, and floating-point operations.

Figure 10 shows the accuracy of YOLOv5n and TC-YOLOv5 in detecting various types of floating debris. Regardless of the types of floating debris, the detection accuracy of TC-YOLOv5 is better than that of YOLOv5n, especially the three types of plastic garbage, ball and leaf have the most significant improvement. It can be concluded that TC-YOLOv5 has excellent generalizability. The above two algorithms have very low detection accuracy in detecting branches, which may be caused by the large differences in the shape and size of branches in the dataset.

Figure 11 visualizes several examples of YOLOv5n and TC-YOLOv5 detecting floating debris. For simple scenes, YOLOv5n has some missed and false detections, which don't occur when using the TC-YOLOv5. For some complex scenes containing branches, TC-YOLOv5 has fewer missed detections than YOLOv5n. Therefore, TC-YOLOv5 outperforms YOLOv5n in detecting floating debris for realistic scenes.

We use the same hyper-parameters as YOLOv5n when training the TC-YOLOv5 model. The optimization algorithm uses stochastic gradient descent (SGD) with an initial learning rate of 0.01 and a learning rate momentum of 0.937. The network training process of YOLOv5n and TC-YOLOv5 is

Table 2 The ablation experiment results

Methods	Model pruning	CBAM	Transformer	$mAP_{@0.5}$	$mAP_{@0.5:0.95}$	Model size (KB)	GFLOP (s)	FPS_{GPU}
YOLOv5n [8]	—	—	—	0.817	0.536	3819	4.2	149
M1	✓	—	—	0.815	0.531	3649	3.8	178
M2	✓	✓	—	0.827	0.537	3683	3.8	164
M3	✓	—	✓	0.833	0.54	3648	3.7	159
TC-YOLOv5	✓	✓	✓	0.842	0.545	3683	3.7	147

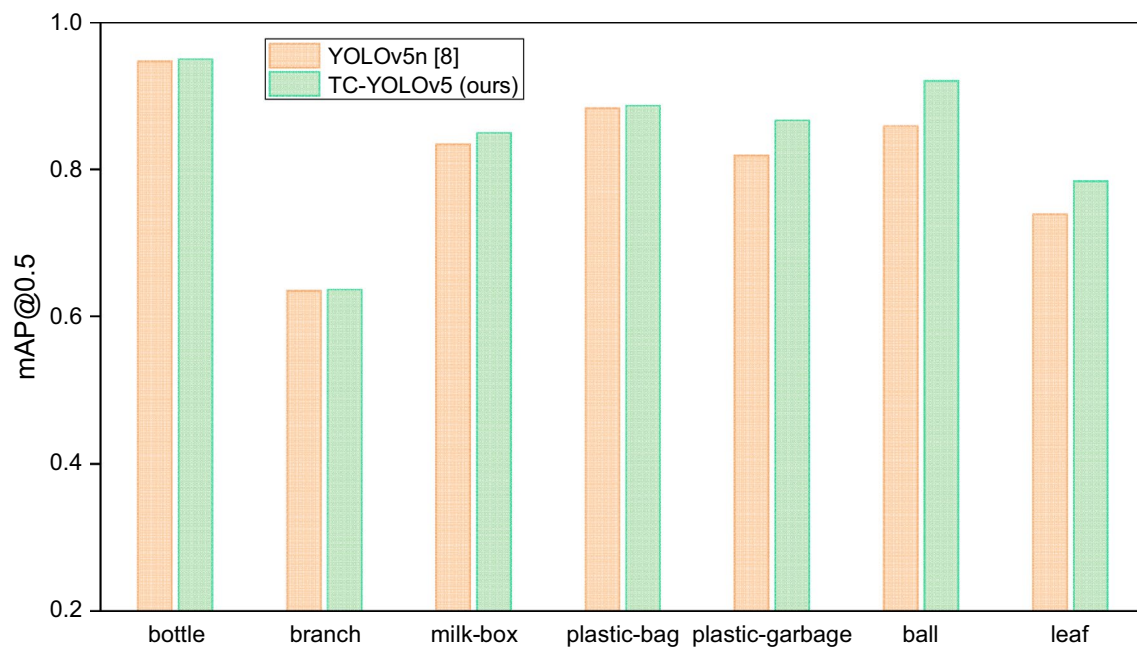


Fig. 10 Comparison of YOLOv5n and TC-YOLOv5 in the detection accuracy of various floating debris

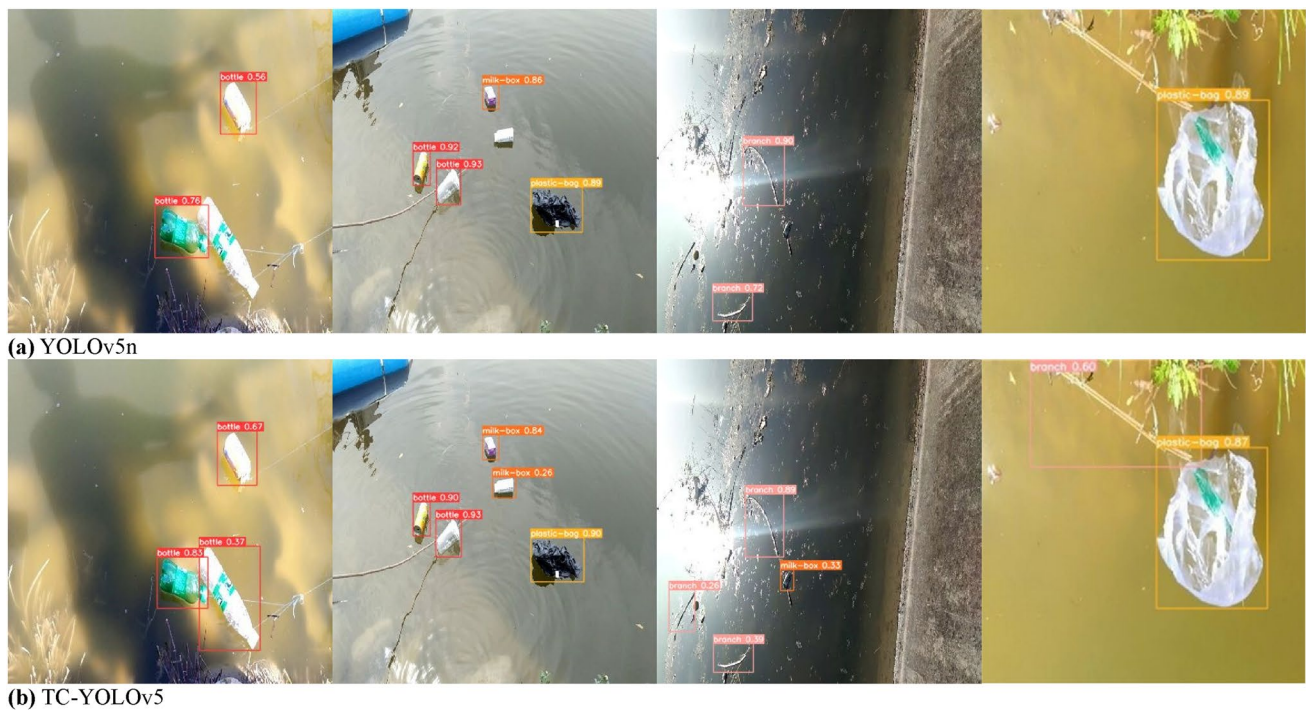


Fig. 11 Comparison of detection results between YOLOv5n and TC-YOLOv5

shown in Fig. 12. In the early stage of model training, the accuracy of YOLOv5n is slightly higher than TC-YOLOv5. In subsequent epochs, the accuracy of YOLOv5n remains almost constant, while the accuracy of TC-YOLOv5 is

increasing and exceeds YOLOv5n. It can be seen that YOLOv5n has a faster convergence speed, and TC-YOLOv5 has higher accuracy when converging.

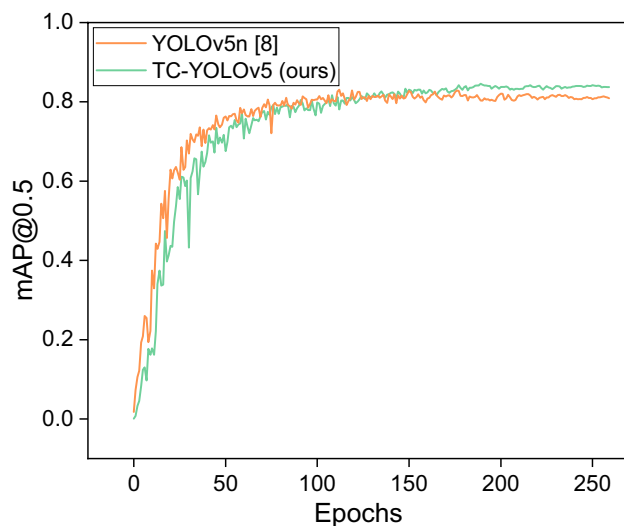


Fig. 12 mAP@0.5 curve in training phase of YOLOv5n and TC-YOLOv5

4.5 Performance comparison with state-of-the-art models

To better evaluate the performance of the TC-YOLOv5 algorithm, we compare it with the YOLOv4 series and YOLOv5 series algorithms for some common metrics in detecting floating objects on the water surface. The YOLOv4 series of algorithms include YOLOv4 and YOLOv4-tiny, and YOLOv4-tiny can be regarded as the lightweight YOLOv4 algorithm. The YOLOv5 series of algorithms includes YOLOv5n, YOLOv5s, YOLOv5m, YOLOv5l, and YOLOv5x, of which YOLOv5n has the least parameters and YOLOv5x has the most. To compare the performance of these algorithms fairly, we specified the input image size to

be 640*640 at the input side of the model and evaluated the performance of these algorithms on PC and Raspberry Pi 4B. The comparison results are shown in Table 3.

As shown in Table 3, the $mAP_{@0.5}$ result of the YOLOv5l algorithm is 83.5%. Among all the algorithms in the YOLOv5 series, YOLOv5l has the highest $mAP_{@0.5}$, but it is at the expense of the detection speed of the model, and the YOLOv4 algorithm has the same drawback. Although the YOLOv5n algorithm has a fast speed, it has a low detection accuracy and cannot accurately detect floating debris. The TC-YOLOv5 algorithm achieves a balance of accuracy and speed, the $mAP_{@0.5}$ result of the TC-YOLOv5 algorithm is 84.2%, which is better than the optimal algorithm (YOLOv5l) by 0.7%, and the $mAP_{@0.5:0.95}$ result of TC-YOLOv5 algorithm is 54.5%, which is better than the optimal algorithm (YOLOv5s) by 0.6%. Since Raspberry Pi 4B has very few computational resources, the computational load of the model needs to be considered. The floating-point operations of our proposed algorithm are 3.7 GFLOPs, which is 0.5 GFLOPs lower than the optimal YOLOv5n of the YOLO series, and the parameters and model size of TC-YOLOv5 algorithm are also better than all models of the YOLOv4 series and YOLOv5 series. The detection speed of our proposed algorithm is second only to YOLOv5n. The execution time of the Raspberry Pi 4B is only one millisecond slower than that of YOLOv5n, which we think may be because the model integrates CBAM and transformer block to increase the inference time of the model, but does not increase the floating-point operations of the model. In conclusion, the overall performance of TC-YOLOv5 algorithm is better than that of other YOLO series algorithms. It considers the accuracy, speed and resource occupation of the model, so it is very suitable for practical applications.

Table 3 The comparison of different object detection algorithms

Model	$mAP_{@0.5}$	$mAP_{@0.5:0.95}$	Parameter (pieces)	Model size (KB)	GFLOP (s)	FPS _{GPU}	Detection time Raspberry (s)
YOLOv4-tiny [1]	0.800	0.499	5,887,976	23,065	16.17	51	2.382
YOLOv4 [1]	0.812	0.529	63,969,996	250,360	141.54	8	34.350
YOLOv5n [8]	0.817	0.536	1,768,636	3819	4.2	149	1.179
YOLOv5s [8]	0.816	0.539	7,029,004	14,148	15.9	58	2.870
YOLOv5m [8]	0.824	0.532	20,877,180	41,299	48.0	40	7.280
YOLOv5l [8]	0.835	0.531	46,140,588	90,763	107.9	24	14.805
YOLOv5x [8]	0.809	0.525	86,213,788	169,166	204.1	13	26.436
TC-YOLOv5	0.842	0.545	1,681,530	3683	3.7	147	1.180

The best results for each column except the first are shown in bold

5 Conclusion

For the recognition scenario of floating debris, this paper proposes TC-YOLOv5 object detection algorithm. By integrating CBAM and transformer block in YOLOv5n, the accuracy is improved and the missed detection rate is reduced. At the same time, the YOLOv5n is pruned to optimize the detection speed and floating-point operations of the model. The experimental results show that the TC-YOLOv5 algorithm can process one image every 1.18 s on Raspberry Pi 4B. Compared with YOLOv5n, the mAP@0.5 increases by 2.5% and the mAP@0.5:0.95 increases by 0.9%, and the floating-point operations are reduced by 0.5 GFLOPs. Compared with other algorithms of the YOLOv4 and YOLOv5 series, the TC-YOLOv5 algorithm can achieve better results in evaluation metrics, such as accuracy, speed, floating-point operations and model size. Finally, the TC-YOLOv5 algorithm can be directly deployed in embedded devices such as Raspberry Pi, which has practical significance and commercial value in fields, such as USVs and UAVs cruises.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

- Bochkovskiy, A., Wang, C.-Y., Liao, H.-Y.M.: YOLOv4: Optimal Speed and Accuracy of Object Detection (2020). <https://doi.org/10.48550/arXiv.2004.10934>
- Cheng, Y., Zhu, J., Jiang, M., Fu, J., Pang, C., Wang, P., Sankaran, K., Onabola, O., Liu, Y., Liu, D., Bengio, Y.: FloW: A Dataset and Benchmark for Floating Waste Detection in Inland Waters. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 10933–10942 (2021). <https://doi.org/10.1109/ICCV48922.2021.01077>
- Dalal, N., Triggs, B.: Histograms of Oriented Gradients for Human Detection. In: 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), pp. 886–893 (2005). <https://doi.org/10.1109/CVPR.2005.177>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Housby, N.: An image is worth 16x16 words: transformers for image recognition at scale (2020). <https://doi.org/10.48550/arXiv.2010.11929>
- Feng, D., Haase-Schutz, C., Rosenbaum, L., Hertlein, H., Glaser, C., Timm, F., Wiesbeck, W., Dietmayer, K.: Deep multi-modal object detection and semantic segmentation for autonomous driving: datasets, methods, and challenges. *IEEE Trans. Intell. Transp. Syst.* **22**, 1341–1360 (2021). <https://doi.org/10.1109/TITS.2020.2972974>
- Girshick, R., Donahue, J., Darrell, T., Malik, J.: Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation. In: 2014 IEEE Conference on Computer Vision and Pattern Recognition, pp. 580–587 (2014). <https://doi.org/10.1109/CVPR.2014.81>
- Girshick, R.: Fast R-CNN, in: 2015 IEEE International Conference on Computer Vision (ICCV), pp. 1440–1448 (2015). <https://doi.org/10.1109/ICCV.2015.169>
- Glenn, J.: yolov5. Git code (2020). <https://github.com/ultralytics/yolov5>
- Guo, M.-H., Xu, T.-X., Liu, J.-J., Liu, Z.-N., Jiang, P.-T., Mu, T.-J., Zhang, S.-H., Martin, R.R., Cheng, M.-M., Hu, S.-M.: Attention mechanisms in computer vision: a survey. *Comput. Vis. Media* **8**, 331–368 (2022). <https://doi.org/10.1007/s41095-022-0271-y>
- He, K., Zhang, X., Ren, S., Sun, J.: Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition. In: Fleet, D., Pajdla, T., Schiele, B., Tuytelaars, T. (Eds.), *Computer Vision – ECCV 2014*, pp. 346–361 (2014). https://doi.org/10.1007/978-3-319-10578-9_23
- He K, Zhang X, Ren S, Sun J.: Deep Residual Learning for Image Recognition. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, Las Vegas, NV, USA, pp 770–778 (2016). <https://doi.org/10.1109/CVPR.2016.90>
- Hearst, M.A., Dumais, S.T., Osuna, E., Platt, J., Scholkopf, B.: Support vector machines. *IEEE Intell. Syst. Their Appl.* **13**, 18–28 (1998). <https://doi.org/10.1109/5254.708428>
- J. Hu, L. Shen and G. Sun.: Squeeze-and-Excitation Networks. In: 2018 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7132–7141 (2018). <https://doi.org/10.1109/CVPR.2018.00745>
- Latif SA, Khairuddin U, Khairuddin ASM.: Development of Machine Vision System for Riverine Debris Counting. In: 2021 6th IEEE International Conference on Recent Advances and Innovations in Engineering (ICRAIE). IEEE, Kedah, Malaysia, pp. 1–6 (2021). <https://doi.org/10.1109/ICRAIE52900.2021.9704016>
- Li, S.-A., Lin, Y.-C., Weng, C.-W., Chen, Y.-H., Lo, C.-H., Yang, M.-H., Hsieh, M.-H., Wong, C.-C.: Circle object recognition based on monocular vision for home security robot. In: 2012 International Symposium on Intelligent Signal Processing and Communications Systems, pp. 258–261 (2012). <https://doi.org/10.1109/ISPACS.2012.6473491>
- Li, X., Tian, M., Kong, S., Wu, L., Yu, J.: A modified YOLOv3 detection method for vision-based water surface garbage capture robot. *Int. J. Adv. Robot. Syst.* **17**, 172988142093271 (2020). <https://doi.org/10.1177/1729881420932715>
- Lin, T.-Y., Dollar, P., Girshick, R., He, K., Hariharan, B., Belongie, S.: Feature Pyramid Networks for Object Detection. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 936–944 (2017). <https://doi.org/10.1109/CVPR.2017.106>
- Liu, S., Qi, L., Qin, H., Shi, J., Jia, J.: Path Aggregation Network for Instance Segmentation. In: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 8759–8768 (2018). <https://doi.org/10.1109/CVPR.2018.00913>
- Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., Berg, A.C.: SSD: Single Shot MultiBox Detector (2016). https://doi.org/10.1007/978-3-319-46448-0_2
- Lowe, D.G.: Object recognition from local scale-invariant features. In: *Proceedings of the Seventh IEEE International Conference on Computer Vision*, pp. 1150–1157 vol.2 (1999). <https://doi.org/10.1109/ICCV.1999.790410>
- Paller G, Élő G.: Towards a Floating Plastic Waste Early Warning System. In: *Proceedings of the 11th International Conference on Sensor Networks. SCITEPRESS - Science and Technology Publications, Online Streaming*, pp. 45–50(2022). <https://doi.org/10.5220/0010894500003118>
- Panwar, N.L., Kaushik, S.C., Kothari, S.: Role of renewable energy sources in environmental protection: a review. *Renew. Sustain. Energy Rev.* **15**, 1513–1524 (2011). <https://doi.org/10.1016/j.rser.2010.11.037>

23. Park J, Woo S, Lee J-Y, Kweon IS.: BAM: Bottleneck Attention Module (2018). <https://doi.org/10.48550/arxiv.1807.06514>
24. Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You Only Look Once: Unified, Real-Time Object Detection. In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 779–788 (2016). <https://doi.org/10.1109/CVPR.2016.91>
25. Redmon, J., Farhadi, A.: YOLO9000: Better, Faster, Stronger. In: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 6517–6525 (2017). <https://doi.org/10.1109/CVPR.2017.690>
26. Redmon, J., Farhadi, A.: Yolov3: An incremental improvement (2018). <https://doi.org/10.48550/arXiv.1804.02767>
27. Ren, S., He, K., Girshick, R., Sun, J.: Faster R-CNN: towards real-time object detection with region proposal networks. IEEE Trans. Pattern Anal. Mach. Intell. **39**, 1137–1149 (2017). <https://doi.org/10.1109/TPAMI.2016.2577031>
28. Wang, C.-Y., Mark Liao, H.-Y., Wu, Y.-H., Chen, P.-Y., Hsieh, J.-W., Yeh, I.-H.: CSPNet: A New Backbone that can Enhance Learning Capability of CNN. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pp. 1571–1580 (2020). <https://doi.org/10.1109/CVPRW50498.2020.00203>
29. Wang Q, Wu B, Zhu P, Li P, Zuo W, Hu Q.: ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). IEEE, Seattle, WA, USA, pp 11531–11539 (2020). <https://doi.org/10.1109/CVPR42600.2020.01155>
30. Woo, S., Park, J., Lee, J.-Y., Kweon, I.S.: CBAM: Convolutional Block Attention Module. In: Ferrari, V., Hebert, M., Sminchisescu, C., Weiss, Y. (Eds.), Computer Vision – ECCV 2018, pp. 3–19 (2018). https://doi.org/10.1007/978-3-030-01234-2_1
31. Xi, P., Guan, H., Shu, C., Borgeat, L., Goubran, R.: An integrated approach for medical abnormality detection using deep patch convolutional neural networks. Vis Comput **36**, 1869–1882 (2020). <https://doi.org/10.1007/s00371-019-01775-7>
32. Zailan, N.A., Mohd Khairuddin, A.S., Khairuddin, U., Taguchi, A.: YOLO-based Network Fusion for Riverine Floating Debris Monitoring System. In: 2021 International Conference on Electrical, Communication, and Computer Engineering (ICECCE), pp. 1–5 (2021). <https://doi.org/10.1109/ICECCE52056.2021.9514096>
33. Zhu X, Lyu S, Wang X, Zhao Q.: TPH-YOLOv5: Improved YOLOv5 Based on Transformer Prediction Head for Object Detection on Drone-captured Scenarios. In: 2021 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW). IEEE, Montreal, BC, Canada, pp 2778–2788 (2021). <https://doi.org/10.1109/ICCVW54120.2021.00312>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Shun Li is currently pursuing the M.S. degree with the School of Computer Science, Wuhan University, Wuhan, China. His current research interests include the computer vision, and deep learning.



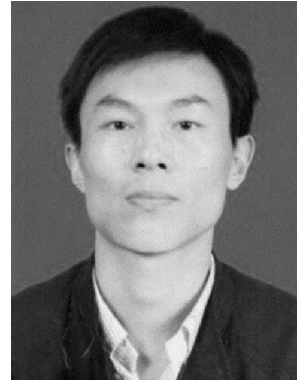
Shubo Liu received the Ph.D. degree in communication and information system from Wuhan University, Wuhan, China, in 2009. He is currently a Full Professor with the School of Computer Science, Wuhan University. His research interests are on multimedia information processing and security, Internet of Things, and edge computing.



Zhaohui Cai received the Ph.D. degree from Wuhan University, Wuhan, China, in 2005. She is currently an Associate Professor with the School of Computer Science, Wuhan University. She is mainly engaged in machine learning, Internet-of-Things technology, and big data analysis and has published many articles and participated in the National 863 program, the National Natural Science Foundation, and the provincial and ministerial scientific and technological research projects.



Yuan Liu is currently pursuing the M.S. degree with the School of Computer Science, Wuhan University, Wuhan, China. His current research interests include the edge computing, and deep learning.



Guoqing Tu received the B.E. degree from the Department of Computer and Application, Hefei University of Technology, Hefei, China, in 1996, and the M.S. and Ph.D. degrees from the School of Computer Science, Wuhan University, Wuhan, China, in 2002 and 2008, respectively. He is currently an Associated Professor with the School of Cyber Science and Engineering, Wuhan University. His current research interests include IoT and its security, embedded system, and hydro-logical monitor-



Geng Chen is currently pursuing the M.S. degree with the School of Computer Science, Wuhan University, Wuhan, China. His current research interests include the machine learning, and computer vision.

ing system.